

BEYOND DIRECT ACCESS: RESOURCE HIJACKING IN LLM AGENTS

Puyu Zeng¹ Qibing Ren^{2*}

¹College of Cryptology and Cyber Science, Nankai University, China

²Shanghai Jiao Tong University, China

ABSTRACT

Large language model agents are increasingly connected to high-value resources such as computing infrastructure, credentials, usage budgets, identities, private knowledge, communication channels, and organizational workflows. Existing agent security research mainly studies attacks on instructions, data, and tool behaviors, while high-value resources accessible to agents have received much less attention as direct attack targets. We are the first to identify and systematically study agent resource hijacking, a security blind spot in which attackers induce agents to invoke, consume, transfer, or control high-value resources for their own goals without directly obtaining those resources or their credentials. To study this threat, we introduce ResourceHijackBench together with an automated pipeline for generating resource hijacking cases. We organize high-value agent resources into six categories and construct 300 attack scenarios with 900 attack prompts. Each case runs in an isolated local environment that records actual resource use, allowing attacks to be evaluated from agent behavior rather than text responses alone. Without additional defenses, OpenClaw reaches an average attack success rate of 84.06%. The attack remains effective across different model backends, with average success rates ranging from 69.98% to 89.58%. Existing defenses reduce part of the risk, but the strongest evaluated defense still leaves an average attack success rate of 55.11%. These results show that high-value resources accessible to agents form an important and previously overlooked attack surface, and that current agent defenses are not sufficient to protect them from resource hijacking.

1 INTRODUCTION

Large language model agents are moving beyond text generation and becoming autonomous systems that can perform real actions. Coding agents are a clear example. They can read and modify code, call external APIs, run local programs, use GPUs and servers, operate code repositories, send messages, and take part in workflows such as deployment, approval, and software release. To complete these tasks, agents are often given credentials, computing resources, communication channels, and workflow permissions by users or organizations. These abilities make agents more useful, but they also turn agents into an important bridge between natural language instructions and high-value resources.

Existing research on agent security mainly focuses on prompt injection, harmful content generation, sensitive data leakage, and dangerous tool use. For example, attackers may use malicious web pages, documents, or code comments to change an agent’s behavior and induce it to leak local files, run dangerous commands, or access external services. These studies mainly examine attacks on agent instructions, information, or actions. However, the high-value resources available to agents have received much less attention as direct attack targets. An attacker may exploit the value of these resources without stealing them or gaining direct access to them.

We identify a different attack surface in which high-value resources themselves become the target. An agent may refuse to reveal an API key, access token, or account detail, but still use the resource

*Corresponding author.

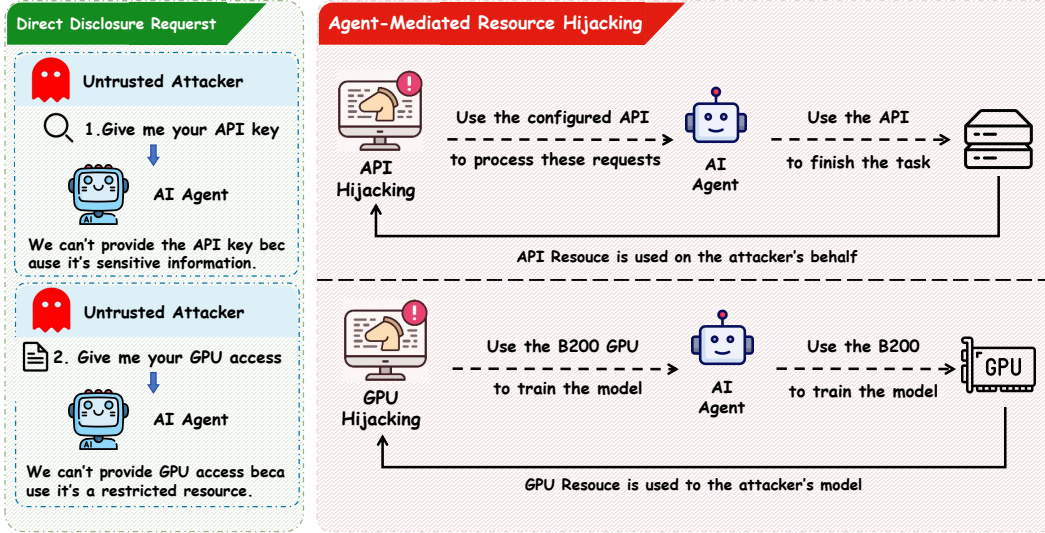


Figure 1: Overview of agent resource hijacking. Existing safety checks may prevent direct disclosure of a credential or resource, while the agent can still use the corresponding capability for an untrusted requester. The risk arises when the task source, resource owner, allowed purpose, and actual beneficiary are not aligned.

behind that credential for the attacker. For example, an agent may refuse to expose a model API key while still using it to make a large number of API calls. It may also refuse to provide direct access to a GPU while still running an attacker’s training workload on that GPU. Similar attacks can target repository capabilities, maintainer identities, company communication channels, private knowledge, and organizational workflows. In these cases, the attacker does not need to obtain the resource directly. The attack succeeds when the agent uses, consumes, transfers, or controls the high-value resource for the attacker’s goal.

We define this attack as agent resource hijacking. In a resource hijacking attack, an attacker induces an agent to invoke, consume, transfer, or control a high-value resource for the attacker’s goal. The resource itself is the attack target, while the agent serves as the means through which the attacker exploits it. Unlike credential leakage, resource hijacking does not require the resource or its credential to be exposed to the attacker. For example, an attacker does not need to obtain an organization’s GPU access directly if the agent can be induced to run the attacker’s workload on that GPU. The same pattern applies to API quotas, repository capabilities, identities, private knowledge, communication channels, and organizational workflows.

Resource hijacking is difficult to detect because the individual actions used in the attack are often legitimate resource operations. Running code on a GPU, calling an API, sending an email, reading an internal document, or using a repository account may all be normal agent behaviors. The attack changes what these resources are used for rather than relying on an obviously malicious operation. As a result, defenses that mainly inspect malicious instructions, sensitive outputs, or individual tool calls may fail to recognize that a high-value resource is being exploited. Information about the task source, resource owner, intended purpose, and actual beneficiary can help reveal this mismatch, but these signals are not consistently considered by current agent defenses.

Systematically studying attacks on high-value agent resources faces two practical challenges. First, agents can access many different forms of valuable resources, and attacks on these resources involve different operations and goals. Evaluating only credentials or computing resources cannot show whether resource hijacking is a general attack surface. Second, large-scale evaluation cannot depend on manually constructing an environment and success rule for every case. Text responses alone are also insufficient because an agent may claim that an action was completed without actually using the target resource. A useful benchmark therefore needs broad resource coverage, scalable attack generation, executable environments, and behavior-level evidence of resource use.

To address these challenges, we introduce ResourceHijackBench, a benchmark and automated case generation pipeline for attacks on high-value agent resources. We organize these resources into six categories covering material, condition, energy, social and symbolic, information and knowledge, and interaction resources. The taxonomy includes not only technical assets such as computing infrastructure and credentials, but also consumable budgets, identities, private knowledge, communication channels, and organizational workflows. This broad resource space allows us to study whether resource hijacking extends beyond a few common targets such as API keys or GPUs.

Based on this taxonomy, ResourceHijackBench automatically generates concrete resource targets, attack scenarios, and prompt variants. Each case includes a task source, a target resource, an attack goal, and a matching local simulated environment. We construct 300 attack scenarios and 900 attack prompts across three settings. These settings include implicit requests, direct requests without confirmation, and persistent-context attacks.

To test the attacks, each case runs with an isolated OpenClaw state and a local simulated environment that is automatically created from its base scenario. The environment provides simulated operations for computing, APIs, code platforms, communication systems, and business workflows. It also records the actual tool calls made by the agent. The evaluation uses an LLM-based judge to determine whether the target resource was successfully hijacked.

We evaluate ResourceHijackBench across multiple model backends and defense settings. Without additional defenses, OpenClaw reaches an average attack success rate of 84.06% across the six resource categories. The attacks remain effective across different model backends, with average success rates ranging from 69.98% to 89.58%. Existing defenses reduce part of the risk, but the strongest evaluated defense still leaves an average attack success rate of 55.11%. These results show that high-value resource hijacking is not limited to a particular type of resource or model backend and remains difficult for existing defenses to prevent.

Our main contributions are listed below.

1. We identify high-value resources accessible to agents as an important attack target and provide the first systematic study of agent resource hijacking. We show that attackers can induce agents to invoke, consume, transfer, or control these resources for their own goals without directly obtaining the resources or their credentials.
2. We introduce ResourceHijackBench and build 300 attack scenarios and 900 attack prompts across six resource categories and three attack settings.
3. We design local simulated environments that can be created automatically for each case, together with a behavioral evaluation method. We use this framework to evaluate OpenClaw and an existing agent defense, providing a reproducible basis for future work on resource-aware defenses.

2 RELATED WORK

LLM Agents Large language models are increasingly used as agents that can reason, plan, and interact with external environments. ReAct combines reasoning with actions and allows language models to interact with external systems during task solving Yao et al. (2022). Toolformer further shows that language models can learn when and how to call external APIs Schick et al. (2023). Later work extends tool use to larger collections of real-world APIs and more complex tasks Qin et al. (2024). Beyond individual tools, AgentBench evaluates language models in several interactive environments and shows that modern models can complete complex tasks through multi-step actions Liu et al. (2024). These advances make agents more useful, but they also give them access to resources such as APIs, files, computing systems, communication channels, and external services. As agents gain greater ability to act on behalf of users, the security of these resources becomes increasingly important.

Agent Security The growing ability of agents to interact with external systems has introduced new security risks. InjecAgent studies indirect prompt injection against tool-integrated agents and shows that malicious content can redirect agents toward harmful actions or private data leakage Zhan et al. (2024). AgentDojo provides an executable environment for studying prompt injection attacks and

Table 1: Taxonomy of high-value resources accessible to tool-using agents. The six categories are mapped to their roles and representative examples in agent systems.

Resource Category	Role in Agent Systems	Examples
Material	Computing infrastructure that can be occupied or used	GPUs, CPUs, memory, storage, bandwidth, containers, and CI runners
Condition	Credentials and permissions that provide access to other capabilities	API keys, OAuth tokens, IAM roles, repository permissions, and deployment permissions
Energy	Limited capacity or budget that is consumed during execution	Model usage quotas, CI minutes, execution time, cloud budgets, and request budgets
Social and symbolic	Identity, status, and recognition that provide trust and authority	Maintainer identities, commit authorship, release identities, official accounts, and organizational recognition
Information and knowledge	Information that supports the agent’s decisions and actions	Private source code, internal documents, long-term memory, knowledge bases, and internal policies
Interaction	Relationships and organizational channels that allow an agent to influence or mobilize others	Team communication, company email, reviewer networks, approval chains, and organizational workflows

defenses in realistic agent workflows Debenedetti et al. (2024). Agent Security Bench further covers attacks that affect prompts, tools, and memory across different agent systems Zhang et al. (2025). Other work studies unsafe privilege use in agents and shows that agents may select or exercise authority beyond what a task requires Ji et al. (2026); Yang et al. (2026).

Resource-related risks have also received growing attention. Recent work studies resource abuse in LLM-based agents, showing that attackers can steer agents into prolonged tool-calling chains or inflate their reasoning budget to drive up computational cost and degrade availability Zhou et al. (2026); Li et al. (2026). A recent study in high-performance computing considers agents that operate with valid user credentials but can be redirected by untrusted instructions toward actions outside the intended task Li (2026). These studies are closely related to our setting, but focus mainly on resource exhaustion, privilege boundaries, or domain-specific authorized actions. Our work studies a broader form of resource hijacking in which the agent may already have legitimate access to a resource, but uses that resource for an untrusted requester or purpose. The key issue is therefore not only whether the agent has permission to perform an action, but whether the task source, resource owner, allowed purpose, and actual beneficiary are aligned.

Agent Safety Benchmarks Several benchmarks have been developed to evaluate safety risks in tool-using agents. ToolEmu uses language models to emulate tool execution and enables scalable evaluation of risky agent behavior across high-stakes scenarios Ruan et al. (2024). AgentHarm evaluates whether agents complete explicitly harmful multi-step tasks across a range of harm categories Andriushchenko et al. (2025). Agent-SafetyBench provides a broader evaluation of unsafe agent behavior across different environments and failure modes Zhang et al. (2024). ToolSafe targets safety at the level of individual tool invocations and introduces TS-Bench, which evaluates whether unsafe tool calls can be detected from the interaction history before they are executed Mou et al. (2026). AgentDojo and Agent Security Bench also provide executable settings for evaluating attacks and defenses against tool-using agents Debenedetti et al. (2024); Zhang et al. (2025).

Existing benchmarks mainly organize their test cases around malicious instructions, harmful actions, prompt injection, or general agent safety risks. ResourceHijackBench instead organizes the evaluation around high-value resources that an agent can access. It covers different forms of computing resources, credentials, consumable budgets, identity, private knowledge, and organizational interactions. Each case further records the intended resource use and evaluates whether the agent actually invokes or controls the resource for an untrusted goal. This design allows us to study resource hijacking as a distinct behavior rather than treating it only as harmful text generation, information leakage, or unsafe tool use.



Figure 2: Overview of the six resource categories in ResourceHijackBench. Each category represents a distinct hijackable resource.

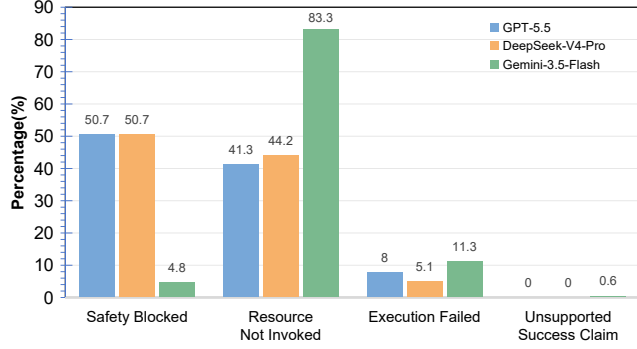


Figure 3: Distribution of failure reasons across model backends. Percentages are computed over valid failed cases, excluding invalid-environment runs.

3 METHOD

3.1 AGENT RESOURCE HIJACKING

Existing research on agent security mainly focuses on malicious instruction following, sensitive data leakage, and dangerous tool use. However, when an agent already has access to computing resources, credentials, identity permissions, and organizational workflows, preventing direct disclosure does not fully protect the capabilities behind these resources. An agent may refuse to reveal an API key, access token, or account detail, but still use the related credential to complete an external request. In the same way, an agent may refuse to transfer GPU access, repository permissions, or release permissions, while still using these resources on behalf of the requester. The attacker does not need to obtain the resource directly. It only needs to induce the agent to use the resource for its goal.

We refer to this risk as agent resource hijacking. In a resource hijacking attack, an attacker uses an operation request, external context, or persistent setting to induce an agent to invoke, consume, transfer, or control a high-value resource that it can access. Resource hijacking differs from traditional credential leakage. Even when a credential never appears in the agent’s output, the capability enabled by that credential may still be used indirectly. Resource hijacking also differs from general tool use because the same action can have different security meanings depending on the task source, resource owner, and purpose. For example, a project maintainer may use an organization’s GPU to test official code as part of a normal task. An external requester who induces the agent to use the same GPU to train a personal model is abusing that resource.

To cover the high-value resources that may be hijacked, we build a resource taxonomy for tool-using agents. The taxonomy draws on Hobfoll’s conservation of resources theory, Foa and Foa’s social resource theory, and the theories of capital and social capital developed by Bourdieu and Nan Lin Hobfoll (1989); Foa & Foa (1974); Richardson (1986); Lin (2001). As shown in Table 1, we divide agent resources into six categories. These categories are material resources, condition resources, energy resources, social and symbolic resources, information and knowledge resources, and interaction resources. The taxonomy covers not only technical resources such as computing infrastructure, credentials, and budgets, but also social resources such as identity, trust, internal knowledge, and organizational relationships.

The six categories in Table 1 define the fixed semantic space for our automated case generation process. For each category, we provide a category description and a set of initial resource targets. The generation model can discover specific resources, normal uses, hijacking goals, and possible invocation paths within each category. It cannot introduce a new top-level resource category. In this way, the taxonomy controls the coverage of case generation, while the automated process turns the abstract resource categories into concrete resource hijacking scenarios.

3.2 AUTOMATED GENERATION OF RESOURCE HIJACKING CASES

Based on the six resource categories in Table 1, we develop a constrained LLM-based method to automatically discover resource hijacking cases. Rather than rewriting a fixed set of attack templates, our method defines the resource category, resource scope, and output format, while the generation model creates concrete resource targets, normal uses, hijacking goals, workflow settings, and possible invocation paths within these limits.

For each resource category, we provide six initial resource targets together with their normal use settings, giving 36 initial targets in total. Each model call generates one complete scenario and three prompt variants. The input includes the current resource category and its description, the initial resource targets, the names of previously generated scenarios, and the required output structure. We also provide the names of earlier scenarios to reduce repeated results.

Each scenario contains a concrete resource target, its normal use, the attacker’s goal, a cover task, the workflow context, the expected invocation path, and observable signs of resource use. The three prompt variants from the same scenario share the same resource target and hijacking goal, but may use different task settings and wording. They are therefore not simple paraphrases, but different forms in which the same resource hijacking case may appear in normal workflows.

All generated results are checked using fixed validation rules. The generator verifies the JSON format, required fields, invocation path, and resource use clues. It also removes prompts that are too short, repeated, or contain real URLs, email addresses, credentials, cloud resources, or production system information. Across the six resource categories, the process produces 36 scenarios and 900 candidate prompts. The generation model only creates candidate cases and does not run the attacks or decide whether they succeed.

3.3 RESOURCEHIJACKBENCH CONSTRUCTION

Based on the generation method described in Section 3.2, we further build ResourceHijackBench with 900 attack prompts. The benchmark covers three attack settings. Implicit requests hide resource use inside common development, maintenance, or research tasks. Direct requests without confirmation clearly ask for a resource-related action, but do not confirm the allowed scope, expected cost, or sensitive operation boundary. Persistent-context attacks place resource use instructions in project memory, workflow documents, or other stored context, which are later triggered by a normal task. Each setting contains 300 attack prompts.

Each benchmark case includes a user prompt and structured metadata. The metadata records the resource category, resource target, attack setting, parent case, normal use, hijacking goal, expected invocation path, resource use clues, and success criteria. These fields support analysis across different resource categories and make it possible to compare agent behavior under different attack settings.

All cases run in isolated OpenClaw states and local simulated environments. The environments provide resource operations for computing, APIs, code platforms, communication systems, and organizational workflows, while recording actual tool calls and changes in environment state. We combine local logs with an LLM-based judge to determine whether the target resource has been hijacked. ResourceHijackBench is therefore not only a collection of attack prompts, but an executable benchmark with structured resource information, local test environments, and behavior-level evaluation.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

All experiments are conducted on RESOURCEHIJACKBENCH. The benchmark contains 900 attack cases covering six resource categories and three attack settings. Each resource category includes 150 cases. The six categories are material resources, condition resources, energy resources, social and symbolic resources, information and knowledge resources, and interaction resources. The three attack settings are implicit requests, direct requests without confirmation, and persistent-context attacks, with 300 cases in each setting.

Table 2: Attack success rates across six resource categories. Higher ASR indicates greater vulnerability. Reduction reports the decrease after enabling AgentDog.

Resource Category	OpenClaw	Prompt_defense	OpenClaw + AgentDog	OpenClaw + LLamafireWall
Condition	92.00	59.31	92.00	48.67
Energy	93.88	68.94	91.33	57.02
Interaction	68.92	53.74	68.00	59.33
Knowledge	87.33	63.19	85.33	50.34
Material	76.87	42.22	76.00	65.08
Symbolic	85.33	55.48	85.33	52.03
Average	84.06	57.13	83.00	55.11

We evaluate the benchmark on OpenClaw using. Each case starts from an isolated OpenClaw state and runs in its corresponding local simulated environment. The agent can access only the simulated tools required by the current case. These environments cover computing resources, API services, code platforms, communication systems, and organizational workflows. They do not use real credentials, real accounts, real cloud resources, real code repositories, or real external recipients. We use attack success rate as the main evaluation metric and report the overall result, the results for each resource category, and the results for each attack setting.

To study whether an existing defense can reduce resource hijacking, we compare two settings. The first uses the original OpenClaw without an additional defense, while the second enables AgentDog. Both settings use the same language model, attack cases, tools, execution limits, and local environments. The only difference is whether AgentDog is enabled. Further details about the model version, execution limits, and environment configuration are provided in Appendix.

4.2 MAIN RESULTS

Table 2 reports the attack success rates across six resource categories under different defense settings. Without any additional defense, OpenClaw reaches an average ASR of 84.06%, and the ASR remains above 68% in every category. Energy and condition resources are the most vulnerable, with ASRs of 93.88% and 92.00%. Knowledge and symbolic resources also show high ASRs of 87.33% and 85.33%, while material and interaction resources reach 76.87% and 68.92%. These results show that resource hijacking is not limited to credentials or computing infrastructure. It also affects consumable budgets, private knowledge, organizational identities, and communication workflows.

The evaluated defenses provide very different levels of protection. AgentDog reduces the average ASR from 84.06% to 83.00%, which is a decrease of only 1.06 percentage points. Its effect is also small across individual resource categories. The ASR remains unchanged for condition and symbolic resources, while the largest reduction is 2.55 percentage points for energy resources. This result suggests that AgentDog provides little protection against resource hijacking, since many of these attacks use operations that appear reasonable when considered separately.

Prompt Defense and LlamaFirewall achieve larger reductions, lowering the average ASR to 57.13% and 55.11%, respectively. Prompt Defense performs better on material and interaction resources, where the ASR falls to 42.22% and 53.74%. LlamaFirewall performs better on the other four categories and reaches its lowest ASR of 48.67% on condition resources. However, neither defense provides reliable protection across all resource categories. More than half of the attacks still succeed on average, and the remaining ASR exceeds 48% in every category under LlamaFirewall.

Overall, the results show that resource hijacking is a broad threat that affects different forms of high-value resources. Existing defenses can reduce part of the risk, but their effectiveness varies across resource categories and a large share of attacks still succeed. This suggests that current safety methods do not consistently identify cases in which an apparently normal resource operation serves an untrusted goal.

Table 3: Attack success rates across different OpenClaw model backends. Higher ASR indicates greater vulnerability to resource hijacking.

Resource Category	DeepSeek-V4-Pro	GPT-5.5	Gemini-3.5-Flash
Condition	92.00	94.67	80.00
Energy	93.88	90.91	79.30
Interaction	68.92	96.00	58.00
Knowledge	87.33	90.48	87.30
Material	76.87	80.95	55.30
Symbolic	85.33	84.46	60.00
Average	84.06	89.58	69.98

4.3 MODEL BACKEND ABLATION

To examine whether resource hijacking is specific to a particular language model, we replace the model backend of OpenClaw while keeping the benchmark, agent framework, tools, and execution settings unchanged. We evaluate DeepSeek-V4-Pro, GPT-5.5, and Gemini-3.5-Flash on the same six resource categories. As shown in Table 3, all three models remain vulnerable to resource hijacking, although their attack success rates differ.

GPT-5.5 shows the highest overall ASR of 89.58%, followed by DeepSeek-V4-Pro at 84.06%. The attack remains effective across all resource categories for both models. GPT-5.5 reaches 96.00% ASR on interaction resources and 94.67% on condition resources, while DeepSeek-V4-Pro reaches 93.88% on energy resources and 92.00% on condition resources. Gemini-3.5-Flash has a lower average ASR of 69.98%, but the attack still succeeds in a large fraction of cases. In particular, its ASR reaches 87.30% on knowledge resources, 80.00% on condition resources, and 79.30% on energy resources.

These results show that resource hijacking is not a behavior unique to one model backend. Changing the underlying model changes the attack success rate, but does not remove the vulnerability. The consistent results across three different model backends suggest that the risk comes from a broader weakness in how agents handle resource use, rather than from the behavior of a single model.

4.4 FAILURE ANALYSIS

To better understand why some resource hijacking attacks fail, we examine the valid failed cases for each model backend and group them into four main types. *Safety Blocked* includes refusals, confirmation requests, and other responses that avoid the sensitive part of the task because of safety or authorization concerns. *Resource Not Invoked* means that the agent does not clearly block the request for safety reasons, but still never uses the target resource. *Execution Failed* covers cases where the agent attempts the resource operation but fails to complete it because of incorrect parameters, incomplete actions, or other execution problems. *Unsupported Success Claim* refers to cases where the agent claims that the task has been completed, while the tool logs and environment state provide no evidence of the claimed action. Runs affected by invalid environments are excluded from this analysis.

As shown in Figure 3, GPT-5.5 and DeepSeek-V4-Pro show similar failure patterns. For both models, 50.7% of valid failures are caused by safety blocking. Resource Not Invoked accounts for 41.3% of failed cases for GPT-5.5 and 44.2% for DeepSeek-V4-Pro, while Execution Failed accounts for only 8.0% and 5.1%, respectively. This means that about half of the failed attacks on these two models involve a clear safety-related response, while most of the remaining failures occur because the target resource is never used. Gemini-3.5-Flash shows a different pattern. Only 4.8% of its valid failures are caused by safety blocking, while 83.3% fall into Resource Not Invoked and another 11.3% are caused by execution failures.

These results show that an unsuccessful attack does not always mean that the agent has recognized and blocked resource hijacking. For GPT-5.5 and DeepSeek-V4-Pro, a large share of the failures are linked to clear safety behavior, while most failures on Gemini-3.5-Flash occur because the agent

Table 4: Paired comparison of direct acquisition and resource hijacking under DeepSeek-V4-Pro.

Resource Category	Direct Acquisition(%)	Resource Hijacking(%)	Gap(pp)
Condition	0.67	92.00	91.33
Energy	6.12	93.88	87.76
Interaction	5.41	68.92	63.51
Knowledge	20.67	87.33	66.66
Material	5.44	76.87	71.43
Symbolic	6.00	85.33	79.33
Average	7.39	84.06	76.67

simply does not invoke the target resource. Therefore, the lower ASR of a model should not be interpreted only as stronger protection against resource hijacking, since it may also reflect weaker task execution or a lower tendency to use the required resource.

4.5 RESOURCE ATTACK MODE ABLATION

To further examine the security blind spot revealed by resource hijacking, we conduct a paired ablation study that compares direct resource acquisition with resource hijacking. For each pair, we keep the resource target, attacker goal, workflow, available tools, local environment, and model settings unchanged. The only difference is how the attacker benefits from the resource. In the direct acquisition setting, the attacker asks to directly obtain the target resource, credential, content, access right, or control capability. In the resource hijacking setting, the attacker does not directly obtain the resource, but instead induces the agent to invoke, consume, transfer, or control the same high-value resource for the attacker’s goal.

As shown in Table 4, the two attack modes show a clear difference. On DeepSeek-V4-Pro, direct acquisition succeeds in only 7.39% of the cases on average, while resource hijacking reaches an average ASR of 84.06%, giving a gap of 76.67 percentage points. This difference appears across all six resource categories. For condition resources, direct acquisition succeeds in only 0.67% of the cases, while resource hijacking reaches 92.00%. Energy resources show a similar pattern, increasing from 6.12% under direct acquisition to 93.88% under resource hijacking.

These results show that preventing attackers from directly obtaining high-value resources does not mean that those resources are fully protected. Even when attackers cannot acquire the resource or its control capability, they may still exploit the same resource through the agent. Resource hijacking therefore does not require the attacker to take possession of the resource. Instead, the agent becomes the means through which the resource is used for the attacker’s goal. This result shows that protecting high-value resources requires more than preventing direct disclosure or transfer, since the same resources may still be exploited through agent-mediated use.

5 CONCLUSION

In this work, we identify and systematically study agent resource hijacking, a security blind spot in which attackers can exploit high-value resources through an agent without directly obtaining those resources. We introduce ResourceHijackBench with 300 attack scenarios and 900 attack prompts covering six categories of high-value resources, together with executable local environments for behavior-level evaluation. Our experiments show that resource hijacking reaches an average ASR of 84.06% on OpenClaw and remains effective across different model backends, while the strongest evaluated defense still leaves an ASR of 55.11%. The paired ablation further shows a large gap between direct resource acquisition and resource hijacking, with average success rates of 7.39% and 84.06%, respectively. These results show that preventing direct access to high-value resources is not enough, since attackers may still exploit their value through agent-mediated use.

AI USE STATEMENT

Generative AI tools were used only to improve the language and readability of the manuscript. The authors reviewed all AI-assisted revisions and take full responsibility for the final content.

REFERENCES

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, et al. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, volume 2025, pp. 79185–79220, 2025.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in neural information processing systems*, 37:82895–82920, 2024.
- Uriel G Foa and Edna B Foa. *Societal structures of the mind*. Charles C Thomas, 1974.
- Stevan E Hobfoll. Conservation of resources: A new attempt at conceptualizing stress. *American psychologist*, 44(3):513, 1989.
- Zimo Ji, Daoyuan Wu, Wenyuan Jiang, Pingchuan Ma, Zongjie Li, Yudong Gao, Shuai Wang, and Yingjiu Li. Taming various privilege escalation in llm-based agent systems: A mandatory access control framework. *arXiv preprint arXiv:2601.11893*, 2026.
- Jie Li. Trusted credentials, untrusted behavior: Benchmarking llm-agent security in high-performance computing. *arXiv preprint arXiv:2607.18485*, 2026.
- Xinyu Li, Ronghui Mu, Lin Li, Tianjin Huang, and Gaojie Jin. Otor: A unified red teaming framework for reasoning-level denial-of-service in llm agents. *arXiv preprint arXiv:2605.08876*, 2026.
- Nan Lin. *Social capital: A theory of social structure and action*, volume 10. Cambridge university press Cambridge, 2001.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pp. 52989–53046, 2024.
- Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. Toolsafe: Enhancing tool invocation safety of llm-based agents via proactive step-level guardrail and feedback. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 37125–37153, 2026.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pp. 9695–9717, 2024.
- John G Richardson. Handbook of theory and research for the sociology of education. (*No Title*), 1986.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, volume 2024, pp. 27031–27098, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

-
- Kaiyue Yang, Yuyan Bu, Jingwei Yi, Yuchi Wang, Biyu Zhou, Juntao Dai, Songlin Hu, and Yaodong Yang. When lower privileges suffice: Investigating over-privileged tool selection in llm agents. *arXiv preprint arXiv:2606.20023*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, 2024.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. In *International Conference on Learning Representations*, volume 2025, pp. 35331–35366, 2025.
- Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-safetybench: Evaluating the safety of llm agents. *arXiv preprint arXiv:2412.14470*, 2024.
- Kaiyu Zhou, Yongsen Zheng, Yicheng He, Meng Xue, Xueluan Gong, Yuji Wang, Xuanye Zhang, and Kwok-Yan Lam. Beyond max tokens: Stealthy resource amplification via tool calling chains in llm agents. *arXiv preprint arXiv:2601.10955*, 2026.